

معسكر هندسة الذكاء الاصطناعي (AIEP) المؤهل للتوظيف

الأسبوع ٣ - اليوم الثالث



إرشادات الجلسات



أنتم المدرب الأهم

إجاباتكم قد تكون أحياناً أهم من محتوانا لزملائكم، لذلك لا تترددون في الإجابة على بعض، وبرضو ما يمنع تعبّرونا وتجاوبوا على أسئلتنا



ما فيه سؤال غلط

ما يهمّ وش تخصصكم أو مدى خبرتكم، كلنا جينا نتعلم هنا من بعض، لذلك لا تترددون في السؤال، لأنه ما فيه سؤال غلط



o تفاعل = o فائدة

التفاعل أساس الجلسة، خلوكم جاهزين تجاوبون على أسئلة وتشتغلون مع بعض في الجلسة

سّـجّينا بالله ؟

أساسيات PyTorch و Datasets و DataLoaders

الأسبوع ٣ - اليوم الثالث

معسكر هندسة الذكاء الاصطناعي (AIEP)

موضوعات الجلسة

1 ما هي PyTorch؟ وما هو ال Tensor؟

2 أبعاد ال Tensor وشكله

3 إنشاء ال Tensors والتعامل معها

4 ضرب المصفوفات وإعادة التشكيل

5 ال Datasets وال DataLoaders



دقيقة واحدة

نشاط 1

صورة ملوّنة مقاسها ٢٢٤ بكسل طولاً $\times$ ٢٢٤ بكسل عرضاً،
وكل بكسل له ٣ قيم للألوان (أحمر، أخضر، أزرق).

كم رقمًا يخزّنه الكمبيوتر لهذه الصورة الواحدة؟





دقيقة واحدة

نشاط 1

الإجابة:

$$100,028 = 224 \times 224 \times 3 \text{ رقمًا}$$

- الكمبيوتر لا يرى صورة، بل يرى أرقامًا مرتّبة في أبعاد.
- هذا الترتيب اسمه **Tensor**، وهو موضوعنا اليوم.
- وهذه الصورة بالضبط هي Tensor بثلاثة أبعاد: 224 و 224 و 3.

ما هي PyTorch؟ وما هو ال Tensor؟

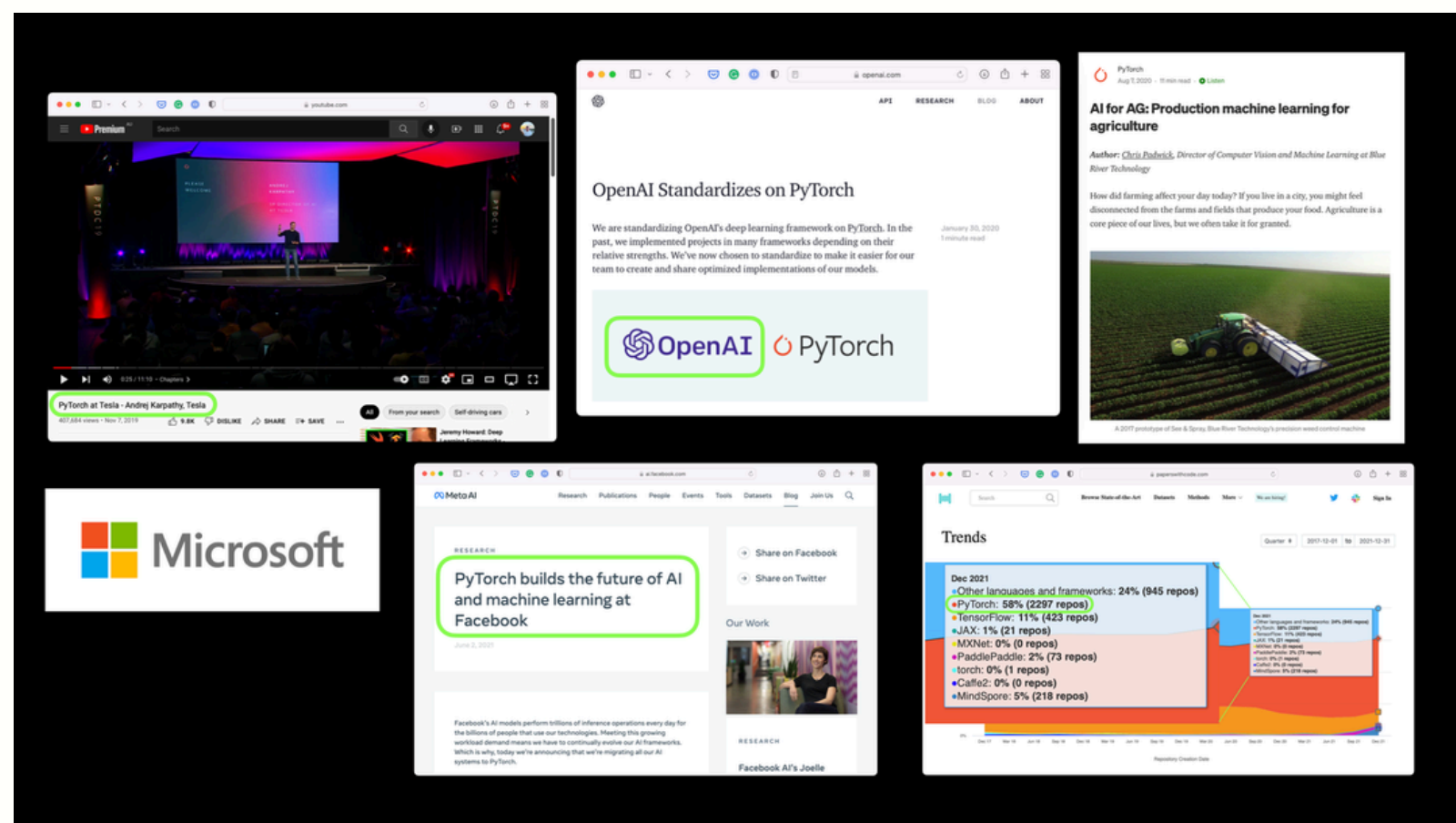
ما هي PyTorch؟

- PyTorch هو إطار عمل مفتوح المصدر لتعلّم الآلة والتعلّم العميق.
- طوّرتها Meta AI في الأصل، وتديرها حاليًا مؤسسة PyTorch Foundation.
- تعتمد على الـ **Tensors**، وهي تشبه مصفوفات NumPy لكنها تعمل على الـ GPU.
- تحسب المشتقات تلقائيًا (**autograd**)، وبهذه الطريقة تتعلّم الشبكات العصبية.



لماذا PyTorch؟

- الأكثر استخدامًا في أبحاث الذكاء الاصطناعي.
- تستخدمها شركات مثل Meta و Tesla و Microsoft و OpenAI لبناء نماذجها.
- كتابتها تشبه Python العادية، فهي سهلة القراءة وتتبع الأخطاء.
- لها منظومة مكتبات ضخمة: **torchvision** للصور، و **torchaudio** للصوت، و **torchtext** للنصوص.



ما هو ال Tensor؟

- ال Tensor وعاء للأرقام مرتبة في بُعد واحد أو أكثر.
- كل البيانات تتحوّل إلى Tensors قبل أن يستخدمها النموذج: الصور والنصوص والصوت والجداول.
- عميل واحد في البنك مثل [العمر، الدخل، الديون] يصبح Tensor فيه ٣ أرقام.
- صورة واحدة تصبح Tensor بثلاثة أبعاد (٢٢٤، ٢٢٤، ٣)، مثل النشاط الأول.
- الشبكة العصبية = ملايين العمليات الحسابية على ال Tensors.

	A	B	C	D	E	F	G	H	I
1	Day of week	1	2	3					
2	Steak sales	3	6	9					
3	Almond butter sales	2	4	5					
4									
5									
6									
7									

استيراد PyTorch



```
import torch
torch.__version__ # shows the installed version
```

- `import torch` يحمل المكتبة؛ اسمها PyTorch لكن في الكود نكتب دائماً `torch`.
- `torch.__version__` يعرض الإصدار المثبت.
- في Colab المكتبة مثبتة مسبقاً، لا نحتاج أي تثبيت.

لاستخدام GPU مجاني في Colab: من قائمة **Runtime** اختر **Change runtime type** ثم **T4 GPU**

أبعاد الـ Tensor وشكله

القيمة المفردة (Scalar)



```
scalar = torch.tensor(7)
scalar          # tensor(7)
scalar.ndim     # 0
scalar.item()   # 7
```

- ال Scalar رقم واحد فقط، وتسمّيه PyTorch: Tensor صفري الأبعاد.
- `scalar.ndim` تعطي عدد الأبعاد، وهو 0 هنا.
- `scalar.item` تُخرج الرقم كعدد Python عادي، وتعمل فقط إذا كان ال Tensor يحتوي قيمة واحدة.

المتجه (Vector)



```
vector = torch.tensor([7, 7])
vector          # tensor([7, 7])
vector.ndim     # 1
vector.shape    # torch.Size([2])
```

- المتجه Tensor أحادي البُعد يحتوي عدة أرقام في صف واحد.
- مثال: متجه فيه الرقمان ٣ و ٢ قد يعني [غرف النوم، دورات المياه] في منزل.
- يحتوي رقمين لكن له **بُعد واحد فقط**.
- **shape** نخبرنا بعدد العناصر في كل بُعد، وهو ٢ هنا.

كيف نعدّ الأبعاد؟

عدّ الأقواس المربّعة من جهة واحدة قبل أول رقم:

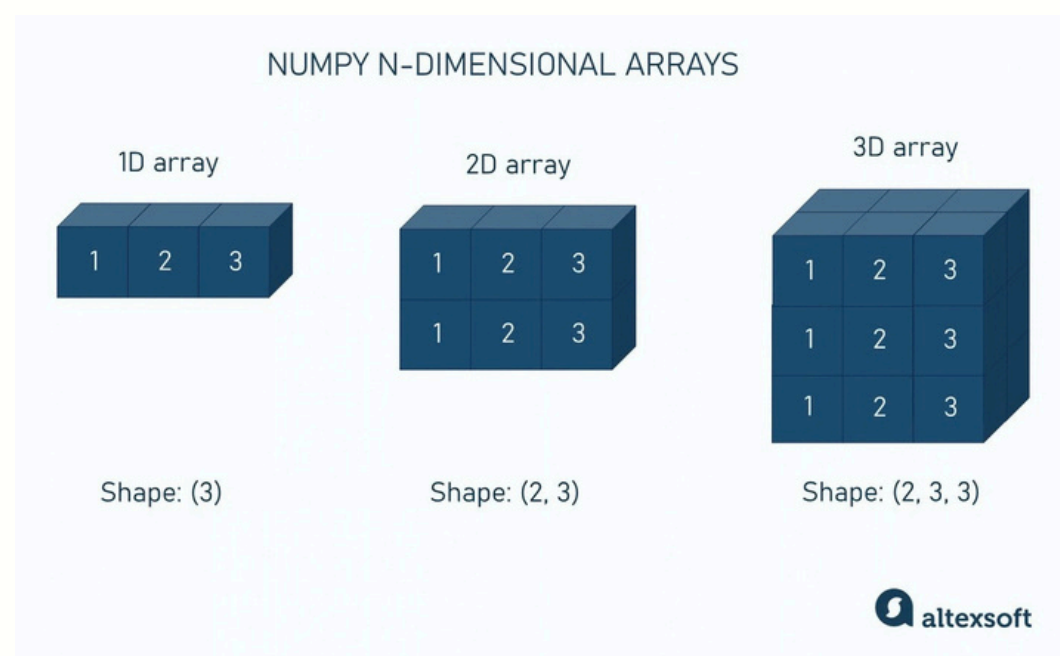


عدد الأبعاد	المثال
· (Scalar)	7
١ (Vector)	[7, 7]
٢ (Matrix)	[[7, 8], [9, 10]]
٣ (Tensor)	[[[1, 2, 3], ...]]

عدد القيم لا يغيّر الأبعاد، التداخل فقط هو الذي يغيّرها، مثل الدمى الروسية.

شكل ال Tensor (shape)

الشكل يخبرنا كم عنصرًا يوجد في كل بُعد.



النوع	الشكل	مثال
Scalar	()	17
Vector	(n,)	[7, 7] → (2,)
Matrix	(الصفوف، الأعمدة)	[[7,8],[9,10]] → (2, 2)
Tensor	(d1, d2, ..., dk)	image → (224, 224, 3)

`shape` = عدد الأرقام داخل ال `shape`
`torch.Size([224, 224, 3])` → `ndim = 3`

المصفوفة (Matrix)



```
MATRIX = torch.tensor([[7, 8],  
                        [9, 10]])  
MATRIX.ndim      # 2  
MATRIX.shape     # torch.Size([2, 2])
```

- المصفوفة ترتّب الأرقام في صفوف وأعمدة، مثل جدول Pandas.
- لها قوسان خارجيان، إذن **بُعدان**.
- الشكل (٢، ٢) يعني صفّين وعمودين.
- بيانات ١٠٠٠ عميل بـ ٥ خصائص = مصفوفة شكلها (١٠٠٠، ٥).

ال Tensor (ثلاثي الأبعاد)

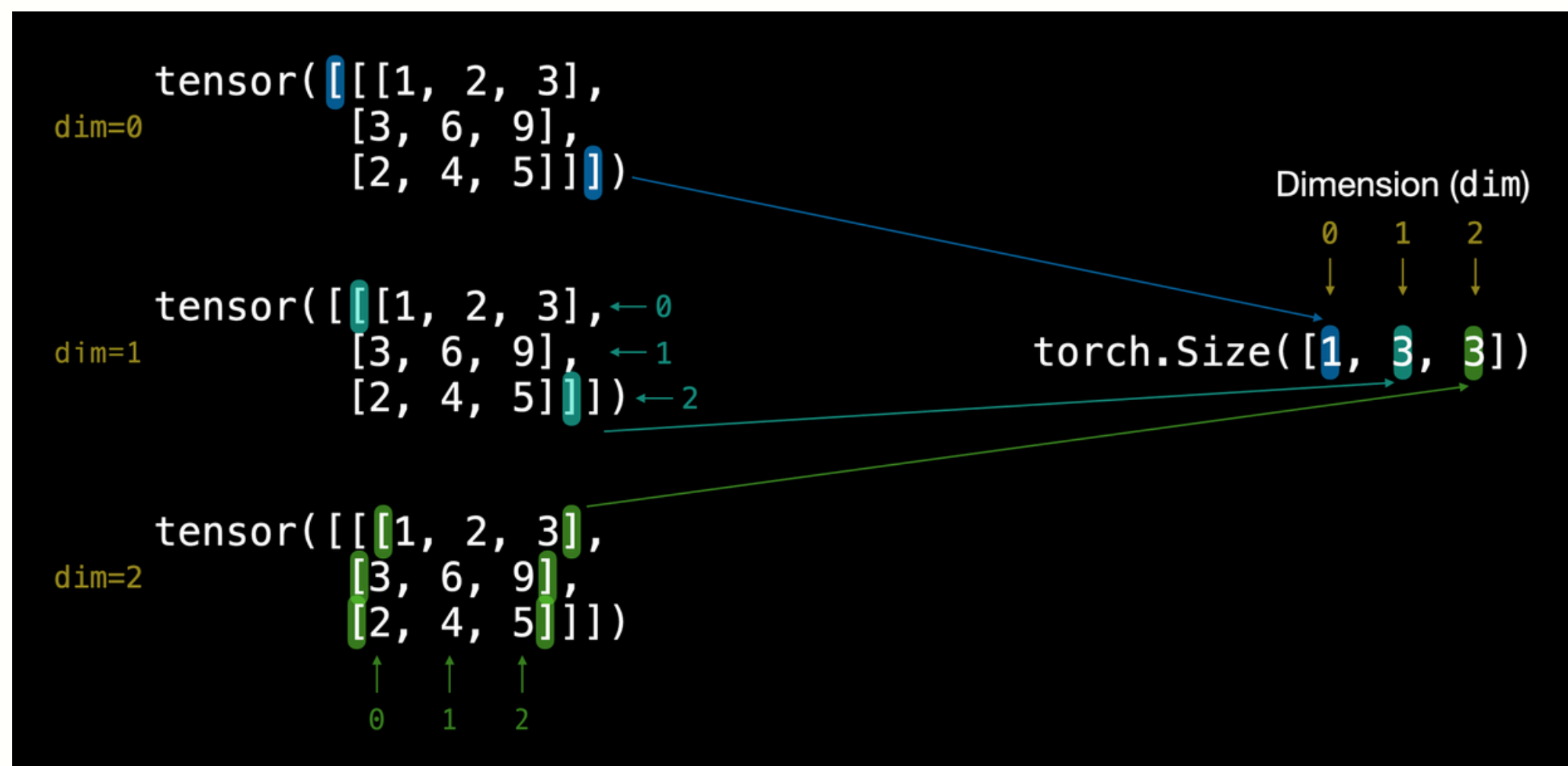


```
TENSOR = torch.tensor([[[1, 2, 3],  
                        [3, 6, 9],  
                        [2, 4, 5]]])  
  
TENSOR.ndim      # 3  
TENSOR.shape     # torch.Size([1, 3, 3])
```

- نقرأ الشكل من الخارج إلى الداخل.
- الشكل (1، 3، 3): كتلة واحدة، فيها 3 صفوف، وكل صف فيه 3 أرقام.
- هذا نفس ترتيب الصورة الملونة، لكن بكتل وبكسلات أكثر.

قراءة الشكل: من الخارج إلى الداخل

كل `dim` يقابل مستوى من الأكواس: `dim=0` هو المستوى الخارجي، و `dim=2` هو الداخلي.



ملخص

Scalar

7

Vector

7

4

or

7

4

Matrix

7

10

4

3

5

1

Tensor

7

4

0

1

1

9

2

3

5

6

8

8

عدد الأبعاد	ما هو؟	الاسم
.	رقم واحد	Scalar
١	صف من الأرقام	Vector
٢	صفوف × أعمدة	Matrix
أي عدد	مصفوفة بعدد n من الأبعاد	Tensor



دقيقتان

نشاط 2

بدون تشغيل الكود:
ما عدد الأبعاد (**ndim**)؟ وما الشكل (**shape**)؟

```
torch.tensor([[[[3, 4], [12, 5], [1, 2], [56, 3]]]])
```





دقيقتان

نشاط 2

الإجابة:

- `ndim = 3` ← هناك ثلاثة أقواس مربعة من جهة واحدة.
- `shape = torch.Size([1, 4, 2])`
- كتلة واحدة، فيها ٤ صفوف، وكل صف فيه رقمان.

إنشاء ال Tensors والتعامل معها

ال Tensors العشوائية



```
random_tensor = torch.rand(size=(3, 4))
random_tensor.dtype      # torch.float32
```

- لماذا عشوائي؟ الشبكة العصبية تبدأ بأرقام عشوائية ثم تعدّلها وهي تتعلّم من البيانات.
- `torch.rand` تنشئ قيمًا عشوائية بين 0 و 1 بأي شكل نختاره عبر `size`.
- النوع الافتراضي `float32`، وهو المعيار في التعلّم العميق.

Tensor بشكل صورة



```
random_image = torch.rand(size=(224, 224, 3))
random_image.shape      # torch.Size([224, 224, 3])
random_image.ndim       # 3
```

- هذه صورة النشاط الأول على شكل Tensor.
- الشكل: [الطول، العرض، قنوات الألوان] = (٢٢٤، ٢٢٤، ٣).
- القيم عشوائية، لكن الشكل هو نفس شكل صورة حقيقية.



الأصفار والآحاد



```
zeros = torch.zeros(size=(3, 4))
ones  = torch.ones(size=(3, 4))
zeros.dtype      # torch.float32
```

- تملأ Tensor بأي شكل بالأصفار أو الآحاد.
- تُستخدم كثيرًا في **الإخفاء (masking)**: نضع أصفارًا مكان القيم التي نريد أن يتجاهلها النموذج.
- هي نفس `np.zeros` و `np.ones` من الأسبوع الماضي.

إنشاء نطاق من الأرقام



```
zero_to_ten = torch.arange(start=0, end=10, step=1)
# tensor([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

- `start` تُحسب، أما `end` فلا تُحسب؛ لذلك النتيجة من ٠ إلى ٩.
- `step` هي المسافة بين كل قيمة والتي بعدها.
- لا تستخدم `torch.range` لأنها قديمة وسيتم إيقافها؛ استخدم دائمًا `torch.arange`.

العمليات الأساسية



```
tensor = torch.tensor([1, 2, 3])
tensor + 10      # tensor([11, 12, 13])
tensor * 10      # tensor([10, 20, 30])
tensor          # tensor([1, 2, 3])
```

- العملية تُطبَّق على **كل عنصر**، وهذا هو ال Broadcasting الذي رأيناه في NumPy.
- لاحظ السطر الأخير: ال Tensor الأصلي **لم يتغيّر!**
- العملية ترجع نتيجة جديدة، لكننا لم نحفظها في متغيّر.

إعادة الإسناد والدوال الجاهزة



```
tensor = tensor - 10      # tensor([-9, -8, -7])
tensor = tensor + 10      # tensor([1, 2, 3])
torch.multiply(tensor, 10) # tensor([10, 20, 30])
```

- لحفظ النتيجة نعيد الإسناد: `tensor = ...`
- لدى PyTorch دوال مثل `torch.add` و `torch.multiply` تؤدي نفس عمل `+` و `*`.
- الرموز أكثر استخدامًا وأسهل في القراءة.

استراحة

10 دقائق

أهلاً بعودتكم

ضرب المصفوفات وإعادة التشكيل

الضرب العنصري مقابل ضرب المصفوفات

لو عندنا Tensor فيه القيم ١ و ٢ و ٣:

العملية	الحساب	الكود	النتيجة
ضرب عنصري	$[1*1, 2*2, 3*3]$	<code>tensor * tensor</code>	$[1, 4, 9]$
ضرب مصفوفات	$1*1 + 2*2 + 3*3$	<code>torch.matmul(tensor, tensor)</code>	14



الفرق هو الجمع:
ضرب المصفوفات يضرب ثم يجمع النواتج،
وهو أهم عملية داخل كل شبكة عصبية.

قاعدتا ضرب المصفوفات

١. البُعدان الداخليان يجب أن يتطابقا

Matrix Multiplication

▶ Multiply

$(2, 3) @ (3, 2)$ ✓ $3 = 3$
 $(3, 2) @ (3, 2)$ ✗ $2 \neq 3$

٢. شكل الناتج يأخذ البُعدين الخارجيين

$(2, 3) @ (3, 2) \rightarrow (2, 2)$
 $(3, 2) @ (2, 3) \rightarrow (3, 3)$

الرمز @ و `torch.matmul` نفس العملية.

لماذا لا نستخدم حلقة for؟



```
# By hand: slow
value = 0
for i in range(len(tensor)):
    value += tensor[i] * tensor[i]

# Built-in: fast
torch.matmul(tensor, tensor)    # tensor(14)
```

- الطريقتان تعطيان ١٤.
- لكن عند قياس وقت التشغيل في Colab نلاحظ أن `matmul` أسرع بكثير، ويكبر الفرق جدًا مع البيانات الحقيقية.
- هذه فكرة **الحوسبة المتجهية**، تجنّب الحلقات على عناصر الـ Tensor قدر الإمكان.



دقيقتان

نشاط 3

أيّ عمليات الضرب التالية تعمل؟ وما شكل الناتج؟



- 1) $(2, 3) @ (3, 4)$
- 2) $(3, 2) @ (3, 2)$
- 3) $(4, 1) @ (1, 4)$
- 4) $(5, 3) @ (5, 3).T$





دقيقتان

نشاط 3

الإجابة:

- ١.  الناتج بشكل (٤، ٢)
- ٢. $\times$ البُعدان الداخليان غير متطابقين (٣ $\neq$ ٢)
- ٣.  الناتج بشكل (٤، ٤)
- ٤.  الناتج بشكل (٥، ٥)؛ لأن التبديل $x.T$ جعل البُعدين الداخليين متطابقين (٣ و ٣)

إعادة التشكيل (reshape) والعرض (view)

```
x = torch.arange(1., 8.)           # shape [7]
x_reshaped = x.reshape(1, 7)       # shape [1, 7]
z = x.view(1, 7)                   # shape [1, 7]
```

- **reshape** تغيّر الشكل دون تغيير القيم، بشرط أن يبقى العدد الكلي نفسه ($V \times 1 = V$).
- **view** تفعل الشيء نفسه لكنها تشارك الذاكرة مع الأصل: تغيير **z** يغيّر **x** أيضًا.
- لماذا يهملنا؟ أغلب أخطاء التعلّم العميق هي **عدم تطابق الأشكال**، وهذه الأدوات تحلّها.

التكديس (stack)



```
x_stacked = torch.stack([x, x, x, x], dim=0)
x_stacked.shape      # torch.Size([4, 7])

torch.stack([x, x, x, x], dim=1).shape
# torch.Size([7, 4])
```

- `stack` تجمع Tensors بنفس الحجم على بُعد جديد.
- `dim=0`: فوق بعض (صفوف) والشكل (٧، ٤).
- `dim=1`: بجانب بعض (أعمدة) والشكل (٤، ٧).
- هكذا بالضبط يجمع الـ DataLoader العينات في دفعة (batch) لاحقًا اليوم.

الضغط والتوسيع: `squeeze` و `unsqueeze`



```
x_resaped.shape          # [1, 7]
x_resaped.squeeze().shape  # [7]
x.unsqueeze(dim=0).shape   # [1, 7]
```

- `squeeze` تحذف الأبعاد الفارغة التي حجمها 1.
- `unsqueeze` تضيف بُعدًا بحجم 1 في المكان الذي نحدده بـ `dim`.
- لماذا؟ النماذج تتوقع بُعد **الدفعة (batch)** في البداية؛
للتنبؤ بعينة واحدة نستخدم `unsqueeze` مع `dim=0`
لتصبح «دفعة فيها عينة واحدة».

إعادة ترتيب الأبعاد (permute)

```
x_original = torch.rand(size=(224, 224, 3))
x_permuted = x_original.permute(2, 0, 1)
x_permuted.shape # torch.Size([3, 224, 224])
```

- `permute` تعيد ترتيب الأبعاد دون تغيير القيم.
- الترتيب (٢، ٠، ١) في `permute` يعني: ضع البعد رقم ٢ أولاً، ثم ٠، ثم ١.
- الصور تُخزن عادةً بترتيب `H, W, C`، لكن نماذج الصور في PyTorch تتوقع `C, H, W`.



ملخص أدوات تغيير الشكل

الدالة	ماذا تفعل؟
reshape	تغيير الشكل مع نفس عدد العناصر
view	مثل reshape لكن يشارك الذاكرة مع الأصل
stack	جمع Tensors على بُعد جديد
squeeze	حذف الأبعاد التي حجمها 1
unsqueeze	إضافة بُعد حجمه 1
permute	إعادة ترتيب الأبعاد

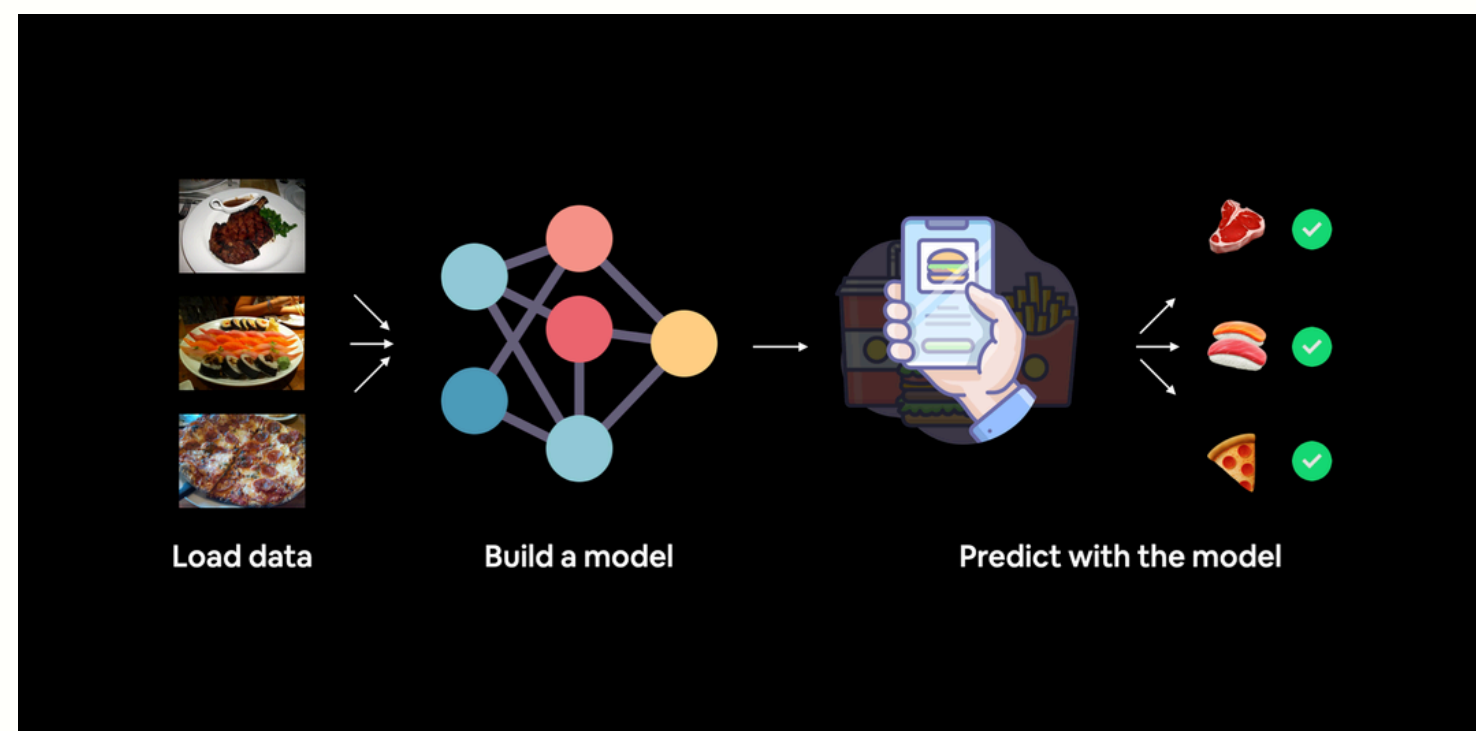
➤ [Introduction to PyTorch.ipynb](#)



ال Datasets و DataLoaders

لماذا نحتاج Datasets و DataLoaders؟

- البيانات الحقيقية قد تكون ملايين الصفوف أو الصور، ولا يمكن تمريرها للنموذج دفعة واحدة.
- يتدرَّب النموذج على مجموعات صغيرة تسمى **الدفعات**، وبالإنجليزي mini-batches.
- يجب **خلط البيانات في كل epoch** حتى لا يحفظ النموذج ترتيبها.
- تقسم PyTorch المهمة إلى أداتين: **Dataset** (ما هي البيانات) و **DataLoader** (كيف تُقدَّم للنموذج).



 مثل المكتبة

DataLoader = أمين المكتبة

يجيب لك الكتب رزمًا، ٣٢ كتابًا كل مرة:

`batch_size=32`

وبترتيب عشوائي:

`shuffle=True`



Dataset = فهرس المكتبة

يعرف عدد الكتب:

`__len__`

ويجيب لك الكتاب رقم *i*:

`__getitem__`

الأمين يطلب من الفهرس العينات واحدة واحدة، ثم يجمعها في دفعة ويقدمها للنموذج.

ما هو ال Dataset؟

- ال Dataset يجاوب على سؤالين:
 - `__len__`: كم عدد العينات؟
 - `__getitem__`: أعطني العينة رقم `i` على شكل (الخصائص، التصنيف).
- لدى PyTorch بيانات جاهزة لكل مجال: `torchvision` للصورة، و `torchaudio` للصوت، و `torchtext` للنصوص.

PyTorch Domain Libraries

"Is this a photo of pizza, steak or sushi?"



TorchVision

"Are these reviews positive or negative?"



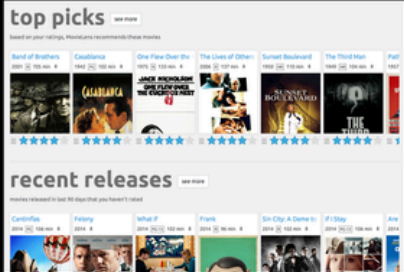
TorchText

"What song is playing?"



TorchAudio

"How do we recommend similar products?"



TorchRec
Source: movielens.org

Different domain libraries contain data loading functions for different data sources

FashionMNIST Dataset جاهز:

```
from torchvision import datasets
from torchvision.transforms import ToTensor

train_data = datasets.FashionMNIST(
    root="data", train=True,
    download=True, transform=ToTensor())

len(train_data)          # 60000
image, label = train_data[0]
image.shape               # torch.Size([1, 28, 28])
```

- ٦٠,٠٠٠ صورة ملابس بالأبيض والأسود مقسمة إلى ١٠ فئات.
- `root` مكان حفظ البيانات، و `train=True` تعني بيانات التدريب.
- الوسيط `transform` مع `ToTensor` يحوّل كل صورة إلى `Tensor`.
- القراءة بالفهرس (مثل العنصر رقم ٠) تستدعي `__getitem__` وترجع زوجًا واحدًا (صورة، تصنيف).
- الشكل (١، ٢٨، ٢٨) بترتيب `C, H, W`: قناة رمادية واحدة و ٢٨×٢٨ بكسل.

Dataset خاص من جدول بيانات

بيانات البنك من نشاط Financial_Risk في الأسبوع الثاني، بعد تنظيفها وترميزها: 



```
import torch
from torch.utils.data import Dataset

class CreditDataset(Dataset):
    def __init__(self, df):
        self.X = torch.tensor(df.drop(columns="risk").values, dtype=torch.float32)
        self.y = torch.tensor(df["risk"].values, dtype=torch.long) # 0=low 1=medium 2=high

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]
```

شرح ال CreditDataset

`__getitem__`

- ترجع خصائص عميل واحد وتصنيفه حسب رقمه `idx`.
- يستدعيها ال DataLoader مرارًا ليبني كل دفعة.

`__len__`

- ترجع عدد العملاء في البيانات.
- يستخدمها ال DataLoader ليعرف كم دفعة سيُنشئ.

`__init__`

- تعمل **مرة واحدة** عند الإنشاء.
- تفصل الخصائص عن عمود `risk` وتحوّلها إلى Tensors.
- الخصائص `float32`، والتصنيفات `long` (أعداد صحيحة) كما يتطلّب التصنيف.

يعمل هذا فقط على بيانات **منظّفة ومرمّزة** (بدون قيم ناقصة أو نصوص)، ولهذا كان الأسبوع الثاني مهمًا.

مجلدات الصور كـ Dataset



```
data/train/  
├─ pizza/  
├─ steak/  
└─ sushi/
```

```
train_data = datasets.ImageFolder(  
    root="data/train", transform=ToTensor())  
train_data.classes # ['pizza', 'steak', 'sushi']
```

- للصور نضع كل فئة في مجلد خاص بها.
- `ImageFolder` تقرأ المجلدات وتستخدم أسماء المجلدات كتصنيفات تلقائيًا.
- لا نحتاج لكتابة class خاص مثل `CreditDataset`.

ما هو ال DataLoader؟

- ال DataLoader يغلف ال Dataset ويقدمه على شكل دفعات.

- `batch_size`: عدد العينات في كل دفعة
- `shuffle`: خلط البيانات في كل epoch؛ نفعله للتدريب ونوقفه للاختبار

- `num_workers`: عمليات في الخلفية تحمّل البيانات بالتوازي
- `drop_last`: حذف آخر دفعة إذا كانت ناقصة



استخدام ال DataLoader



```
from torch.utils.data import DataLoader

train_loader = DataLoader(train_data,
                           batch_size=32, shuffle=True)
test_loader   = DataLoader(test_data,
                           batch_size=32, shuffle=False)

len(train_loader)      # 1875
images, labels = next(iter(train_loader))
images.shape           # torch.Size([32, 1, 28, 28])
labels.shape            # torch.Size([32])
```

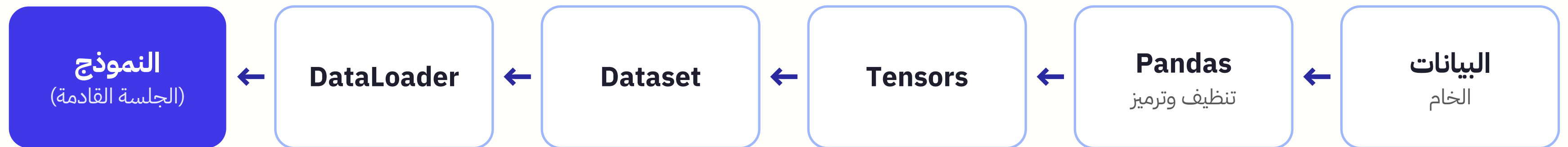
- ٦٠,٠٠٠ صورة ÷ ٣٢ = ١٨٧٥ دفعة في كل epoch.
- الدالتان `next` و `iter` تسحبان أول دفعة فقط لنفحصها.
- ظهر بُعد جديد بحجم ٣٢ في البداية بترتيب `C, batch`، `H, W`؛ ال `DataLoader` كدّس ٣٢ صورة معًا.
- بيانات الاختبار لا نخلطها حتى يبقى التقييم ثابتًا.

مصطلحات أساسية

المصطلح	بالعربي	المعنى
Sample	عينة	صف أو صورة واحدة
Batch	دفعة	عينات تُعالج معًا
Iteration	تكرار	مرور دفعة واحدة عبر النموذج
Epoch	دورة كاملة	مرّ النموذج على كل البيانات مرة واحدة

مثال: ٦٠,٠٠٠ صورة مع دفعة ٣٢ = ١٨٧٥ تكرارًا في كل epoch،
و ٥ epochs = ٩٣٧٥ تكرارًا.

خط المعالجة الكامل

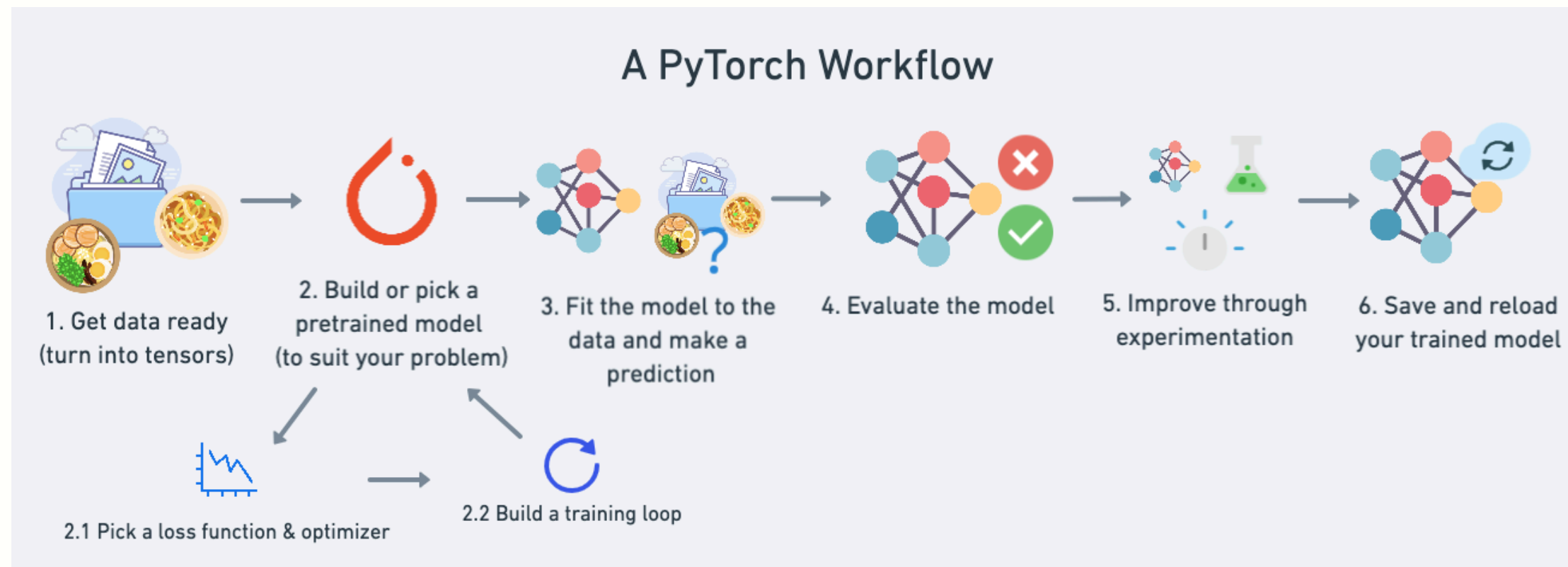


```
for epoch in range(5):  
    for images, labels in train_loader:  
        # model training goes here (next session)  
        pass
```

- الحلقة الخارجية تمرّ على الـ epochs، والداخلية تمرّ على الدفعات.
- هذا الهيكل ذو الحلقتين هو أساس كل كود تدريب في PyTorch ستكتبه.

ماذا بعد؟

- اليوم: جرّنا البيانات ك Tensors وقدمناها للنموذج على شكل دفعات.
- في الجلسات القادمة: نبني النموذج، ونختار دالة الخسارة والمُحسّن، ونكتب حلقة التدريب، ثم نقيّم النموذج ونحفظه.





دقيقتان

نشاط 4

بيانات البنك فيها ١٠٠٠ عميل، واخترنا `batch_size=64`.

١. كم دفعة في كل epoch؟

٢. كم حجم آخر دفعة؟

٣. ماذا يتغير مع `drop_last=True`؟

٤. هل نخلط بيانات الاختبار؟ ولماذا؟





دقيقتان

نشاط 4

الإجابة:

- ١٦.١ دفعة
- ٤٠.٢ عميلًا (١٠٠٠ - ١٥ × ٦٤)
- ١٥.٣ دفعة؛ يُتجاهل آخر ٤٠ عميلًا، ويختلفون في كل epoch بسبب الخلط
- ٤.٤ لا؛ الترتيب لا يؤثر على التقييم، وتثبيته يجعل النتائج قابلة للتكرار

قبل ما نمشي

ايش أهم قرار هندسي بتأخذه
بشكل مختلف بعد اليوم؟
